Principais Comandos do Git ## 1. Configuração e identificação | Comando | Para que serve | |---|---| | `git config --global user.name "Nome"` | Define o nome do usuário | | `git config --global user.email "email@exemplo.com"` | Define o e-mail do usuário | | `git config --list` | Exibe as configurações do Git | ## 2. Criar e obter um repositório | Comando | Para que serve | |---|---| | `git init` | Cria um novo repositório Git na pasta atual | | `git clone URL` | Copia um repositório remoto para o computador | | `git remote -v` | Mostra os repositórios remotos configurados | Exemplo: ```bash git clone https://github.com/usuario/projeto.git ``` ## 3. Verificar o estado do projeto Um dos comandos mais importantes: ```bash git status ``` Mostra arquivos modificados, arquivos novos, arquivos preparados para commit e a branch atual. Ver o histórico: ```bash git log ``` Versão compacta: ```bash git log --oneline ``` ## 4. Adicionar arquivos para o commit Arquivo específico: ```bash git add arquivo.py ``` Arquivos modificados e novos da pasta atual: ```bash git add . ``` Todas as alterações do repositório: ```bash git add -A ``` > Para alunos iniciantes, `git add .` costuma ser suficiente, desde que o resultado seja conferido com `git status`. ## 5. Criar um commit ```bash git commit -m "mensagem do commit" ``` Exemplo: ```bash git commit -m "feat: adiciona geração de relatório PDF" ``` O commit cria um ponto de registro das alterações no histórico local. ## 6. Trabalhar com branches Criar uma branch: ```bash git branch nome-da-branch ``` Criar uma branch e mudar para ela imediatamente: ```bash git switch -c nome-da-branch ``` Exemplo: ```bash git switch -c feat/pdf-report ``` Ver as branches: ```bash git branch ``` Mudar de branch: ```bash git switch main ``` Excluir uma branch local: ```bash git branch -d nome-da-branch ``` ## 7. Sincronizar com o GitHub Atualizar a branch atual: ```bash git pull origin main ``` Enviar commits para o GitHub: ```bash git push origin nome-da-branch ``` Exemplo: ```bash git push origin feat/pdf-report ``` ## 8. Trabalhar com o repositório remoto Adicionar um repositório remoto: ```bash git remote add origin URL ``` Alterar a URL de um remoto: ```bash git remote set-url origin URL ``` ## 9. Desfazer alterações Descartar alterações não commitadas de um arquivo: ```bash git restore arquivo.py ``` Retirar um arquivo do staging, mantendo suas alterações: ```bash git restore --staged arquivo.py ``` > ⚠️ Evite comandos destrutivos, como `git reset --hard`, enquanto estiver aprendendo. Eles podem apagar trabalho que ainda não foi salvo em um commit. ## 10. Tags e versões Criar uma tag: ```bash git tag v1.0.0 ``` Enviar uma tag para o GitHub: ```bash git push origin v1.0.0 ``` Enviar todas as tags: ```bash git push origin --tags ``` # ⭐ Comandos essenciais para o projeto ```text git status git pull git switch git switch -c git add git commit git push git log ``` # 🔄 Ciclo básico de trabalho ```text Trabalhar no código ↓ git status ↓ git add . ↓ git commit -m "..." ↓ git push ↓ GitHub / PR ``` # 🌿 Criando uma nova funcionalidade ```bash git switch main git pull origin main git switch -c feat/nova-funcionalidade ``` Depois de implementar e testar: ```bash git status git add . git commit -m "feat: adiciona nova funcionalidade" git push origin feat/nova-funcionalidade ``` Depois: 1. Acesse o GitHub. 2. Crie um Pull Request. 3. Revise as alterações. 4. Faça o Merge. 5. Volte para `main`. 6. Atualize o repositório: ```bash git switch main git pull origin main ``` # 🧠 Regra importante para o projeto Como o projeto utiliza **OpenSpec + IA + GitHub**, execute: ```bash git status ``` antes e depois das operações importantes. Isso permite verificar o que a IA modificou, quais arquivos foram criados, quais alterações estão prontas para commit e em qual branch você está trabalhando. > **Não trate o Git como uma sequência de comandos para decorar.** Use `git status` para entender o estado atual do projeto antes de tomar a próxima decisão. # 📌 Resumo rápido | Situação | Comando principal | |---|---| | Ver o estado do projeto | `git status` | | Ver histórico | `git log --oneline` | | Atualizar a `main` | `git pull origin main` | | Criar uma branch | `git switch -c nome-da-branch` | | Trocar de branch | `git switch nome-da-branch` | | Preparar alterações | `git add .` | | Criar commit | `git commit -m "mensagem"` | | Enviar para GitHub | `git push origin nome-da-branch` | | Ver branches | `git branch` | | Desfazer alteração local | `git restore arquivo` | | Retirar arquivo do staging | `git restore --staged arquivo` |